Sphincs+ Hash Based Digital Signatures

→ Today's digital signatures (RSA, ECDSA) relay on math problems with hidden algebric structures.

shor's Algorithm breaks

Grover's Algorithm → brute-force hashes slightly faster.

→ Security reduces from $2^n \rightarrow 2^{n/2}$

Preimage - 2^n

birth-day collision - $2^{n/2}$

Quantum

Preimage - $2^{n/2}$

collision - $2^{n/2}$

Basic DS

Alice		Bob
K^-, K^+	$\xrightarrow{K^+}$	K^+

Keychaing

Signing

$S = Sig(m, K^-)$ $\xrightarrow{(m, S)}$ verify

verify $(m, S, K^+) = T/F$

SpHincs+

↳ it is a post-quantum DS scheme build entirely on widely trusted hash function.

Building Blocks

- WOTS+
- Merkle tree
- FORS
- Hypertree

WOTS [Winternitz One-Time Signatures]

Naive hash-based signature

Alice		Bob
$K^- = R$ $K^+ = H(R)$	$\xrightarrow{K^+}$	K^+
signing $Sig(m, K^-) = R$	$\xrightarrow{(R, m)}$	verify
	Attack (R, m')	

→ it completely lacks Message integrity Because Signature s does not dynamically depends on M

WOTS → Hash chain

$R \to H(R) \to H(H(R)) \dots \to H^n(R)$

Toy Example

$H(m) = 1010110\dots$

ω - winternitz parameter

lets consider $\omega = 3$ $[0, 7]$

Protocal for $\omega = 3$

Alice		Bob
$K^- = R$ $K^+ = H^7(R)$	$\xrightarrow{K^+}$	K^+

$m = 101 = 5$

signing

$S = Sig(m) = H^m(R)$

$= H^5(R)$

$\xrightarrow{(m, S)}$ verify

$V = H^{7-S}(S)$

$= T/F$

Attack

$\xrightarrow{(m', S')}$ just increment

WOTS + [checksum]

lets' consider

$\omega = 2$ bit $[0, 1, 2, 3]$

$H(m_{05}) = 1001\dots1$

Message $= [1, 2]$ (M_0, M_1)

Compute checksum

$M_0 = 3 - 1 = 2$

$M_1 = 3 - 2 = 1$

Now new Message

$= [1, 2, 3]$ (SK_1, SK_2, SK_3)

$R_1 \to H^3(R_1) \to R_1^+$

$R_2 \to H^3(R_2) \to K_2^+$

$R_3 \to H^3(R_3) \to K_3^+$

$Sig1 = H^1(R_1)$

$Sig2 = H^2(R_2)$

$Sig3 = H^3(R_3)$

Alice		Bob
K^+_s	$\longrightarrow$	K^+_s
m Compute checksum Create Signature	$\xrightarrow{\text{send}(m, S)}$	Bob receives (m, S) ↓ recompute checksum ↓ Verify hash chains ↓ Compare public keys

In real 256 bit hash

w - window size = 4 bits

chain size $= 2^4 = 16$

Max per digit = 15

$64 \times 15 = 960$

$\log_2(960) = 10\text{bit} \approx 3$

Total $= 64 + 3 = 67$ chains

Merkle Tree

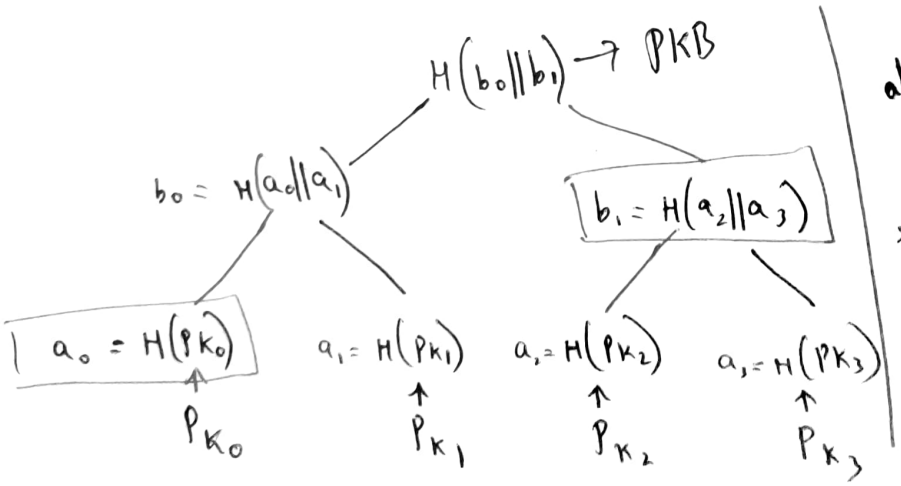
Problem with WOTS+ → Bob needs to store many public keys.

from above we get 67 chains means 67 $PK(K^+)$

lets make only one public key insted of 67 K^+,

$K^+ = H(K_1^+ \| K_2^+ \| K_3^+ \cdots K_{67}^+)$ ↳ This is

One WOTS+ Key (K^+)



alice $\xrightarrow{PKB}$ bob

PKB

$sign(m, x_1) \xrightarrow{S, m, Path}$ verify

→ Merkle tree lets us use many one-time signatures while publishing only one public key.

FORS [Forest of Random Subsets]

→ few-time signature using many small Merkle trees, where each tree signs one part of the message.

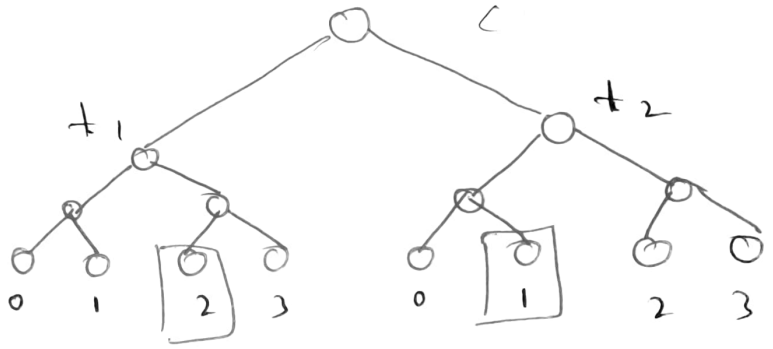
Toy Example

$K = 2$ trees

height $h = 2$

Let message $M = [10\ 01]$ split into 2 parts

2 1



Signature

$[(S_2, \text{Path}_1), (S_1, \text{path}_2)]$

Why "Few Times"

↳ If we sign many messages:

→ You reveal many leaves

→ Eventually attacker learn tree.

Real FORS

$n = 256$ bits

$K = 33$

$h = 6$

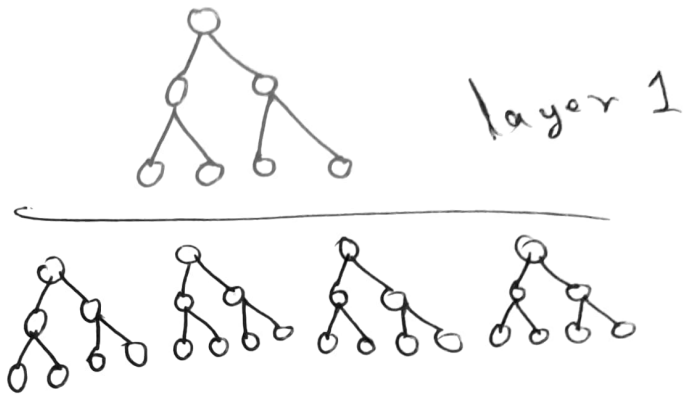
Hyper Tree : → Tree of Merkle Trees

we stack multiple small Merkle Trees in layers.

Toy example :

layers (d) = 2

height per tree (h) = 2



Pseudo Random Function

PRF(Key, input) →

PRF (SK.seed, ADRS)

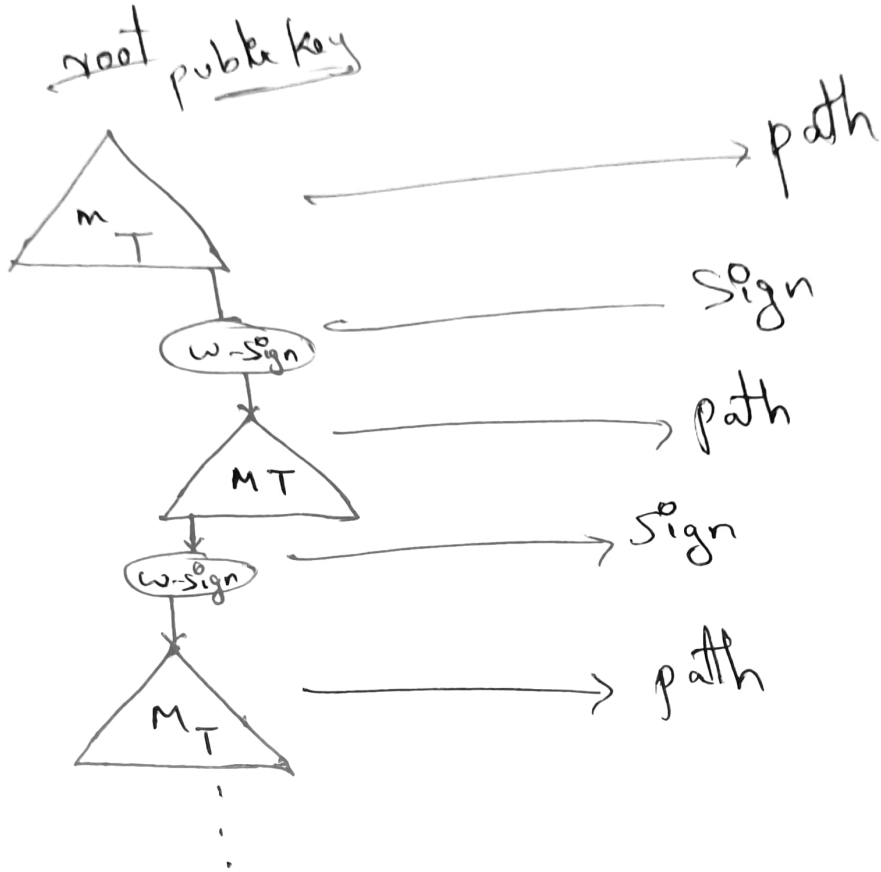
layer no.
Tree index.

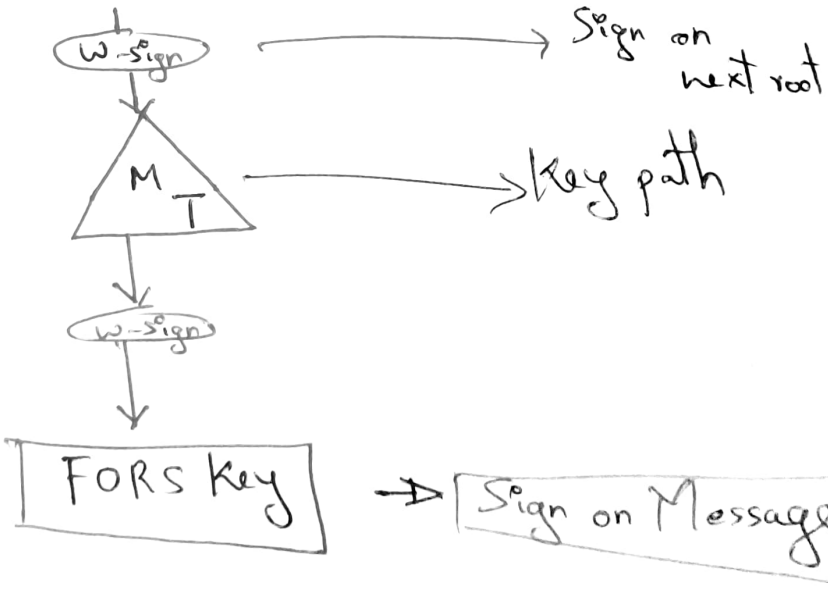
Real → layer = 8 | 17

ht = 4 | 8

total height = 64

Sphincs + Structure





Message → FORS K → WOTS + → WOTS + → Master public key

Sphincs+ setup

Phase 1: Alice Generates Her Keys.

The secret Seeds:

[Secret Key seed] **Seed 1:** Used to magically generate any WOTS+ or FORS Key in the forest on-demand.

[Public seed] **Seed 2:** Used inside the ADRS (Address Structure) to ensure every hash in her tree is unique.

2. **Building the Penthouse:**

→ Alice uses her seeds to calculate only the single top layer of her hypertree.

The public key → The root node of that single top tree is her public key, she publishes this key to the world.

Phase 2: Alice Signs the Contract

The GPS Coordinate:

$$H(m) = \underbrace{\text{chunk1}}_{\text{FORS}} \; \underbrace{\text{chunk2}}_{\text{tree index}} \; \underbrace{\text{chun 3}}_{\text{leaves index.}}$$

climbing the ladder

WOTS+ Key in Layer 0 sign that FORS root
WOTS+ Key in layer 1 sign layer 0.

The Delivery : Signature [30KB list containing the revealed FORS leaves, the WOTS+ signatures, and the Merkle connecting paths)

phase 3 : Bob verifies the signature.

→ Re calculating the GPS

Bottom-up & Final Match with Public key.